

SevaSetu — Technical Architecture Document

Version

Version 2.0 — Hackathon MVP Architecture

Product

SevaSetu — Community Volunteer Coordination Platform

Problem Statement

PS 09 — Community Volunteer Coordination Platform

Core Principle

Right Volunteer. Right Place. Right Impact.

Core Workflow

Need → Match → Schedule → Deploy → Track → Learn

1. Architecture Goals

The architecture must support SevaSetu's central workflow while remaining simple enough to build within the hackathon MVP.

The system must support:

1. Volunteer registration and profiles.
2. NGO profiles and memberships.
3. Public event discovery.
4. Independent/freelance volunteer participation.
5. Event requirements and shifts.
6. Explainable multi-criteria matching.
7. Automated/basic roster generation.
8. Human-controlled roster finalization.
9. Attendance and task completion.
10. Karma calculation and verified contribution history.
11. Reliability tracking.
12. NGO Transparency & Activity Indicators.
13. Event Intelligence Agent.
14. Emergency mobilization as a secondary/demo feature.
15. Notifications.

16. Analytics.

17. Privacy-conscious location matching.

The architecture should also allow SevaSetu to scale to more NGOs, volunteers, events and regions later.

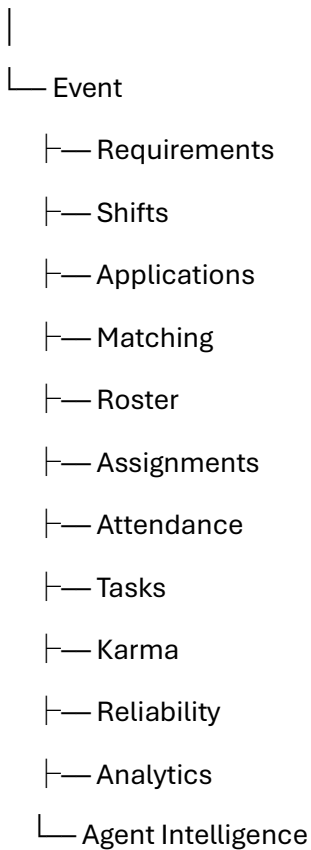
2. Architectural Principles

2.1 Event-Centered Architecture

The Event is the central operational entity.

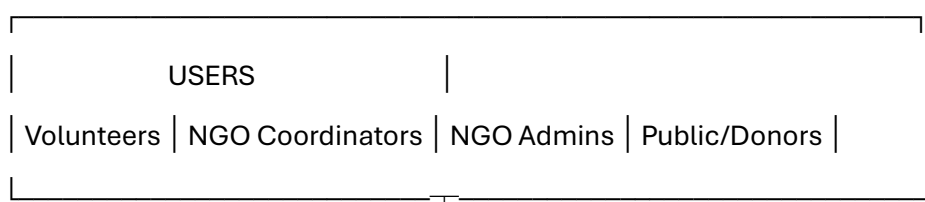
Almost every major workflow is connected to an event:

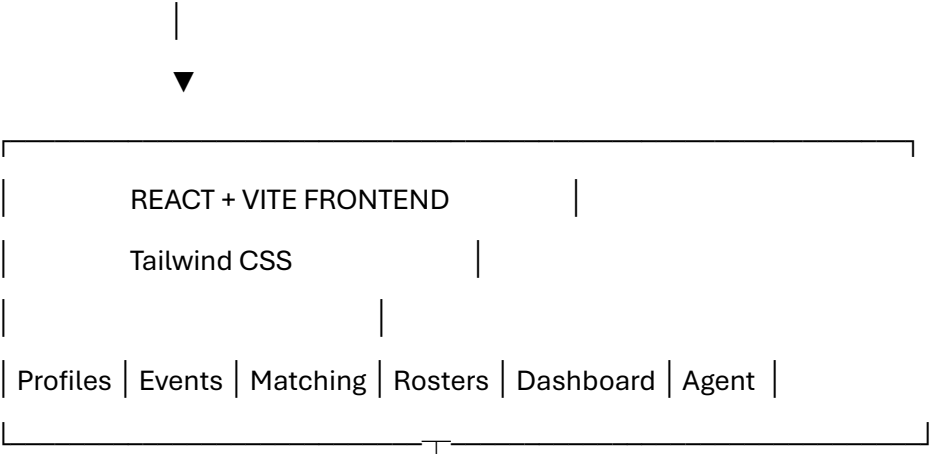
NGO



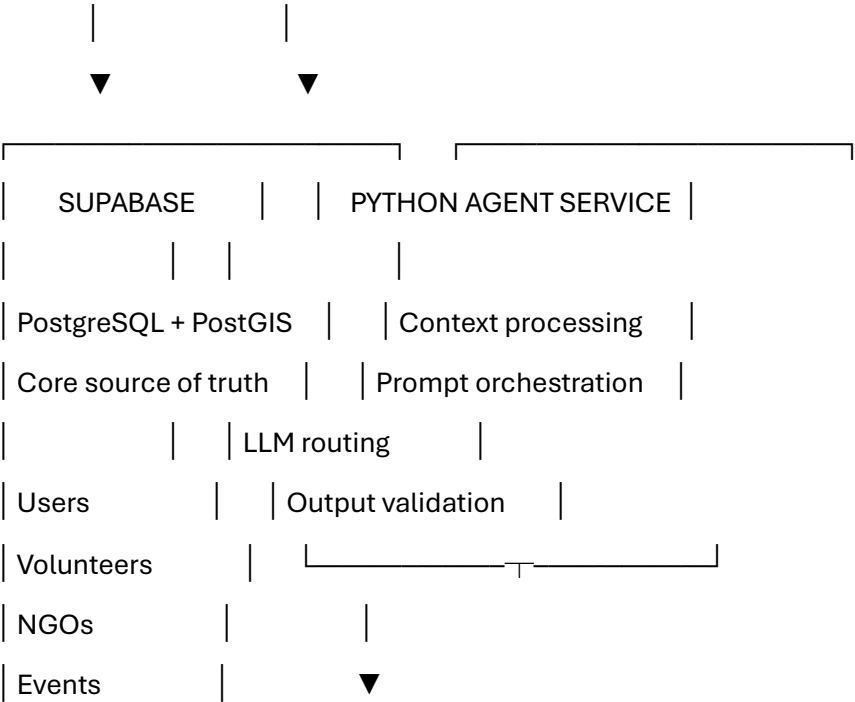
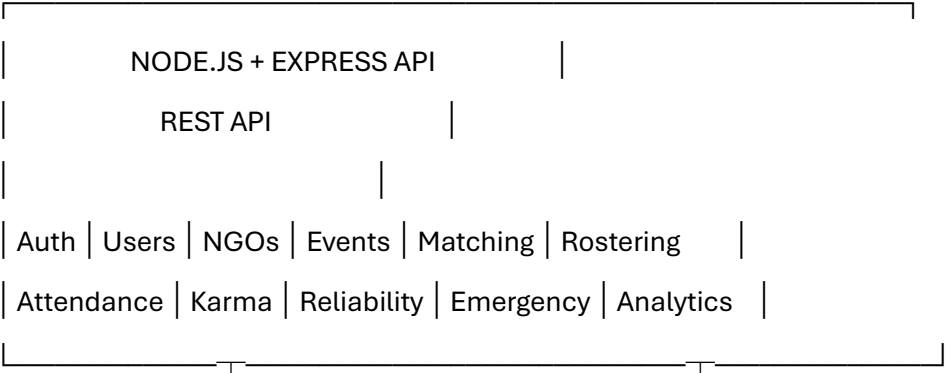
Emergency requests are treated as event-linked urgent operations.

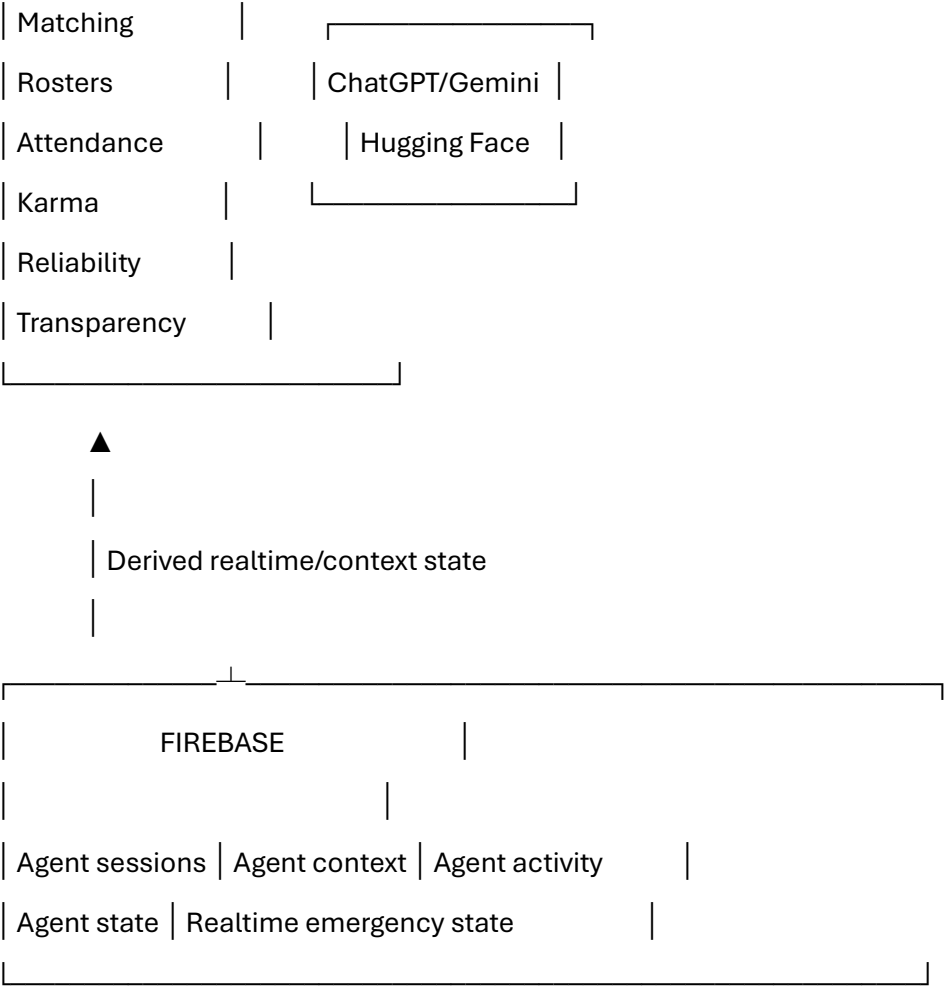
3. High-Level System Architecture





Clerk Session





External Services:

Clerk → Authentication

Maps API → Geocoding + Distance

Email API → Notification fallback

n8n → Workflow automation

Vercel → Frontend/API deployment

4. Recommended Technology Stack

The following stack follows the PRD's specified technical architecture.

4.1 Frontend

HTML

Used as the foundational web technology.

React.js

Recommended for:

- Component-based UI
- Role-specific dashboards
- Event pages
- Volunteer profiles
- NGO management
- Agent interface
- Reusable components

Vite

Used as the React development/build tool.

Benefits:

- Fast development server
- Fast builds
- Simple configuration
- Suitable for a hackathon MVP

Tailwind CSS

Used for:

- Responsive UI
- Consistent design
- Rapid development
- Role-based dashboards
- Event cards
- Tables
- Matching results

The PRD explicitly specifies:

HTML + React.js + Vite + Tailwind CSS.

5. Backend

Node.js

Node.js will run the main SevaSetu application backend.

It is suitable because the application is heavily API-driven and involves:

- User requests
 - Event operations
 - Matching
 - Scheduling
 - Notifications
 - Real-time coordination
 - Database interactions
-

Express.js

Express will provide the REST API layer.

Responsibilities:

- Authentication verification
 - Authorization
 - Input validation
 - Business logic
 - Database operations
 - Matching orchestration
 - Roster generation
 - Emergency workflows
 - Agent context construction
-

6. API Architecture

Use:

/api/v1/

Example:

/api/v1/me

/api/v1/volunteers/me

/api/v1/ngos

/api/v1/events

/api/v1/events/:id/matches

/api/v1/events/:id/roster

/api/v1/events/:id/attendance

/api/v1/karma

/api/v1/emergencies

/api/v1/agent/sessions

All protected endpoints should require:

Clerk Authentication

↓

Role Check

↓

NGO Scope Check

↓

Input Validation

↓

Business Logic

7. Authentication

Clerk

Clerk should handle:

- Registration
- Login
- Sessions
- Identity
- Role claims

The backend verifies the Clerk token on every protected request.

SevaSetu should not implement its own password authentication system.

8. Authorization

There are two levels of authorization.

Platform role

Examples:

volunteer
ngo_coordinator
ngo_admin
platform_admin

NGO scope

An NGO role must also be associated with a specific NGO.

For example:

User A

Role: ngo_coordinator

NGO: Helping Hands

User A should not automatically gain access to another NGO's data.

9. Supabase Architecture

Supabase is the **core source of truth**.

It stores authoritative platform records.

Use Supabase/PostgreSQL for:

- Users
- Volunteer profiles
- Skills
- Certifications
- Availability
- NGOs
- NGO memberships
- NGO affiliations
- Events
- Event requirements
- Shifts
- Applications
- Matching results
- Rosters
- Assignments

- Attendance
- Tasks
- Reliability
- Karma
- Achievements
- Emergency records
- Transparency
- Notifications
- Analytics

The existing SevaSetu technical approach already defines Supabase as the core database and Firebase as Agent-specific data/state.

10. Why Use Two Databases?

Supabase and Firebase have intentionally different responsibilities.

Supabase

Permanent authoritative data.

Examples:

Attendance

Karma

Events

Assignments

Registrations

Reliability

NGO profiles

Volunteer profiles

Firebase

Agent context and ephemeral realtime state.

Examples:

Agent conversations

Agent sessions

Agent context

Agent activity

Emergency live-count state

Temporary Agent state

The important rule is:

Firebase must never become a second source of truth.

If Firebase disappears, authoritative data must still exist in Supabase.

11. Python Agent Service

The Event Intelligence Agent should be deployed separately from the Node/Express API.

Responsibilities:

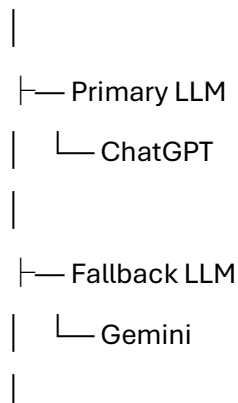
- Context processing
- Prompt construction
- LLM routing
- Historical event analysis
- Preparation recommendations
- Risk suggestions
- Skill recommendations
- Requirement suggestions
- Output validation

The Agent should not directly write to Supabase.

12. AI Model Architecture

Recommended:

Python Agent Service



└ Optional ML

└ Hugging Face

Hugging Face should only be introduced if the application actually needs:

- Embeddings
- Similar-event retrieval
- Classification
- Specialized ML processing

Do not add unnecessary ML complexity to the MVP.

13. Agent Security Boundary

The Agent receives:

Role

Question

Event Context

Relevant Historical Context

Constraints

It returns:

Answer

Basis

Uncertainty

Suggestions

It must NOT have tools for:

- Creating emergencies
- Broadcasting alerts
- Finalizing rosters
- Modifying attendance
- Awarding Karma
- Changing reliability
- Deploying volunteers

The Agent recommends.

Humans decide.

14. Firebase Architecture

Recommended collections:

agentSessions/

agentSessions/{sessionId}/messages/

agentContext/

agentActivity/

agentOutputs/

agentState/

realtime/emergencies/

Firebase documents should contain IDs referencing Supabase records rather than duplicating entire entities.

Example:

```
{
  "sessionId": "session_123",
  "userId": "clerk_user_123",
  "eventId": "event_456",
  "mode": "volunteer"
}
```

Not:

```
{
  "event": {
    "...entire event object..."
  }
}
```

15. Complete Project Folder Structure

Recommended repository structure:

sevasetu/

|

|— apps/

| |

| |— web/

| | |— public/

| | | |— icons/

| | | |— images/

| | | |— favicon.ico

| | |

| | |— src/

| | | |— assets/

| | | |

| | | |— components/

| | | | |— ui/

| | | | |— forms/

| | | | |— events/

| | | | |— volunteers/

| | | | |— ngos/

| | | | |— matching/

| | | | |— roster/

| | | | |— agent/

| | | | |— emergency/

| | | | |— analytics/

| | | |

| | | |— pages/

| | | | |— auth/

| | | | |— volunteer/

| | | | |— ngo/

| | | | |— events/

```
| | | | ├── public/
| | | | └── admin/
| | | |
| | | ├── layouts/
| | | ├── hooks/
| | | ├── services/
| | | ├── api/
| | | ├── store/
| | | ├── utils/
| | | ├── types/
| | | ├── constants/
| | | ├── routes/
| | | ├── App.jsx
| | | └── main.jsx
| | |
| | ├── .env.example
| | ├── index.html
| | ├── package.json
| | ├── vite.config.js
| | └── tailwind.config.js
| |
| ├── api/
| | ├── src/
| | | ├── config/
| | | ├── middleware/
| | | | ├── auth.js
| | | | ├── roles.js
| | | | ├── ngoScope.js
| | | | ├── validation.js
| | | | └── errorHandler.js
```

```
| | | |
| | | |└─ modules/
| | | |└─ users/
| | | |└─ volunteers/
| | | |└─ ngos/
| | | |└─ events/
| | | |└─ matching/
| | | |└─ rostering/
| | | |└─ attendance/
| | | |└─ karma/
| | | |└─ reliability/
| | | |└─ emergency/
| | | |└─ transparency/
| | | |└─ analytics/
| | | |└─ notifications/
| | | |└─ agent/
| | | |
| | | |└─ routes/
| | | |└─ services/
| | | |└─ repositories/
| | | |└─ validators/
| | | |└─ utils/
| | | |└─ db/
| | | |└─ jobs/
| | | |└─ app.js
| | | |└─ server.js
| | |
| | |└─ tests/
| | |└─ unit/
| | |└─ integration/
```

```
| | | └─ security/
| | |
| | └─ .env.example
| | └─ package.json
| |
| └─ agent/
|   └─ app/
|     └─ api/
|       └─ core/
|         └─ context/
|           └─ prompts/
|             └─ models/
|               └─ guardrails/
|                 └─ services/
|                   └─ firebase/
|                     └─ schemas/
|                       └─ main.py
|
|   └─ tests/
|     └─ requirements.txt
|       └─ Dockerfile
|         └─ .env.example
|
| └─ packages/
|   └─ shared-types/
|     └─ validation/
|       └─ constants/
|         └─ matching-config/
|
| └─ supabase/
```

- | ├── migrations/
 - | | ├── 001_extensions.sql
 - | | ├── 002_users.sql
 - | | ├── 003_volunteers.sql
 - | | ├── 004_skills.sql
 - | | ├── 005_certifications.sql
 - | | ├── 006_availability.sql
 - | | ├── 007_ngos.sql
 - | | ├── 008_events.sql
 - | | ├── 009_matching.sql
 - | | ├── 010_rosters.sql
 - | | ├── 011_attendance.sql
 - | | ├── 012_karma.sql
 - | | ├── 013_reliability.sql
 - | | ├── 014_emergency.sql
 - | | ├── 015_transparency.sql
 - | | └── 016_notifications.sql
- | |
- | ├── seed/
 - | | ├── demo_users.sql
 - | | ├── demo_ngos.sql
 - | | ├── demo_events.sql
 - | | ├── demo_volunteers.sql
 - | | └── demo_history.sql
- | |
- | └── functions/
 - |
- └── docs/
 - | ├── architecture/
 - | ├── api/

```
|   ├── database/
|   ├── security/
|   └── agent/
|
|   ├── scripts/
|   ├── seed.js
|   ├── reset-db.js
|   └── generate-types.js
|
|   ├── .github/
|   │   └── workflows/
|   │       ├── frontend.yml
|   │       ├── api.yml
|   │       └── tests.yml
|   │
|   ├── .gitignore
|   ├── README.md
|   └── package.json
```

16. Why This Folder Structure?

The architecture separates the application into three major runtime systems:

web

```
|
|
└── User interface
```

api

```
|
|
└── Core business logic
```

agent

```
|
```

└─ AI intelligence

This prevents AI logic from becoming mixed into core business logic.

For example:

matching/

should contain deterministic matching rules.

While:

agent/

contains natural-language intelligence.

This distinction is important because matching eligibility and ranking must remain deterministic and auditable.

17. Complete Database Schema

The following schema is based on the event-centered data model defined in the PRD.

18. users

Stores the application's identity mapping.

Field	Type	Purpose
id	UUID	Internal user ID
clerk_user_id	TEXT	Clerk identity
email	TEXT	User email
role	ENUM	volunteer / ngo_coordinator / ngo_admin / platform_admin
status	ENUM	active / suspended / disabled
created_at	TIMESTAMP	Creation time
updated_at	TIMESTAMP	Last update

Relationship:

users

|

└─ volunteer_profiles

└─ ngo_members

One Clerk identity corresponds to one SevaSetu user.

19. volunteer_profiles

Stores volunteer-specific information.

Field	Type	Purpose
user_id	UUID	References users
name	TEXT	Volunteer name
photo_url	TEXT	Optional profile image
bio	TEXT	Short description
home_geo	GEOGRAPHY	Private location
max_distance_km	NUMERIC	Maximum travel distance
causes	TEXT[]	Preferred causes
interests	TEXT[]	Volunteer interests
profile_visibility	BOOLEAN	Recruitment visibility
history_visibility	BOOLEAN	Public history setting
notification_preferences	JSONB	Notification settings
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Last update

Location should not be publicly exposed.

20. skills

Master list of skills.

Example:

First Aid

Crowd Management

Teaching

Medical Assistance

Driving

Logistics

Food Distribution

Rescue Support

Fields:

Field	Type
id	UUID
name	TEXT
category	TEXT
active	BOOLEAN

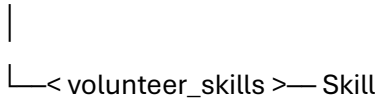
21. volunteer_skills

Many-to-many relationship between volunteers and skills.

Field	Type
volunteer_id	UUID
skill_id	UUID
level	TEXT

Relationship:

Volunteer



22. certifications

Stores volunteer certifications.

Field	Type
id	UUID
volunteer_id	UUID
name	TEXT
issuer	TEXT
expiry	DATE
document_url	TEXT

Field	Type
verified_by_ngo_id	UUID
verification_status	ENUM
created_at	TIMESTAMP

MVP certification model:

Self-declared

↓

Optional NGO verification

Advanced independent verification remains future scope.

23. availability

Stores recurring and exceptional availability.

Field	Type
id	UUID
volunteer_id	UUID
weekday	INTEGER
start_time	TIME
end_time	TIME
recurring	BOOLEAN
start_date	DATE
end_date	DATE
created_at	TIMESTAMP

This data is used by matching and scheduling.

24. ngos

Stores organization information.

Field	Type
id	UUID

Field	Type
name	TEXT
description	TEXT
causes	TEXT[]
region	TEXT
geo	GEOGRAPHY
contact	JSONB
logo_url	TEXT
created_at	TIMESTAMP
updated_at	TIMESTAMP

25. ngo_members

Links users to NGOs.

Field	Type
id	UUID
ngo_id	UUID
user_id	UUID
role	ENUM
created_at	TIMESTAMP

Roles:

ngo_admin

ngo_coordinator

This table determines NGO scope.

26. ngo_affiliations

Represents the relationship between a volunteer and an NGO.

This is deliberately separate from the user account.

A volunteer can be:

Independent

with one NGO while being:

Affiliated

with another NGO.

Field	Type
id	UUID
ngo_id	UUID
volunteer_id	UUID
status	ENUM
invited_at	TIMESTAMP
accepted_at	TIMESTAMP
ended_at	TIMESTAMP

Possible status:

invited

active

ended

rejected

27. events

This is the central table of SevaSetu.

Field	Type
id	UUID
ngo_id	UUID
title	TEXT
description	TEXT
cause	TEXT
venue	TEXT
geo	GEOGRAPHY
start_time	TIMESTAMP

Field	Type
end_time	TIMESTAMP
visibility	ENUM
capacity	INTEGER
status	ENUM
registration_mode	ENUM
created_by	UUID
created_at	TIMESTAMP
updated_at	TIMESTAMP

Visibility:

public

ngo_only

Lifecycle:

Draft

↓

Published

↓

In Progress

↓

Completed

or

Cancelled

28. event_requirements

Defines what an event needs.

Field	Type
id	UUID

Field	Type
event_id	UUID
role	TEXT
headcount	INTEGER
required_skill_ids	UUID[]
required_certifications	TEXT[]
minimum_experience	NUMERIC
created_at	TIMESTAMP

Example:

Role: Medical Support

Headcount: 5

Required skills:

First Aid

Medical Assistance

Required certification:

First Aid Certification

Minimum experience:

1 year

29. shifts

Defines event shifts.

Field	Type
id	UUID
event_id	UUID
requirement_id	UUID

Field	Type
start_time	TIMESTAMP
end_time	TIMESTAMP
headcount	INTEGER

30. applications

Represents volunteer registration.

Field	Type
id	UUID
event_id	UUID
volunteer_id	UUID
status	ENUM
registered_at	TIMESTAMP

Status:

interested

accepted

waitlisted

rejected

cancelled

Add a unique constraint:

(event_id, volunteer_id)

This prevents duplicate registration.

31. match_runs

Stores every matching execution.

Field	Type
id	UUID
event_id	UUID

Field	Type
run_by	UUID
config_snapshot	JSONB
created_at	TIMESTAMP

Why store the configuration?

Because if matching weights change later, SevaSetu can still explain how an older recommendation was generated.

32. match_results

Stores the recommendation generated for each volunteer.

Field	Type
id	UUID
match_run_id	UUID
volunteer_id	UUID
eligible	BOOLEAN
rank	INTEGER
score	NUMERIC
criteria_results	JSONB
explanation	TEXT
created_at	TIMESTAMP

Example:

```
{
  "skills": {
    "status": "met",
    "score": 0.95
  },
  "availability": {
    "status": "met",
    "score": 1
  }
}
```

```
},
"proximity": {
  "status": "met",
  "distance_km": 4.2
},
"experience": {
  "status": "partial"
},
"reliability": {
  "status": "met"
}
}
```

33. Matching Formula

The MVP should use deterministic weighted matching.

Recommended default:

Skills / Certifications 35%

Availability 20%

Proximity 20%

Experience 15%

Reliability 10%

Total:

100%

Important:

Karma is not part of the matching score.

Matching should first apply eligibility gates:

Eligibility

↓

Skills / Certifications

↓

Availability

↓

Proximity

↓

Experience

↓

Reliability

↓

Final ranking

This makes the system explainable.

34. rosters

Stores generated rosters.

Field	Type
id	UUID
event_id	UUID
status	ENUM
generated_at	TIMESTAMP
finalized_at	TIMESTAMP
conflict_report	JSONB
finalized_by	UUID

Status:

draft

final

A roster remains draft until an authorized NGO user finalizes it.

35. assignments

Connects volunteers to shifts.

Field	Type
id	UUID

Field	Type
event_id	UUID
shift_id	UUID
volunteer_id	UUID
type	ENUM
status	ENUM
match_result_id	UUID
created_at	TIMESTAMP

Assignment type:

primary

backup

Status:

proposed

confirmed

declined

deployed

cancelled

36. attendance

Stores verified participation.

Field	Type
id	UUID
assignment_id	UUID
volunteer_id	UUID
checkin_at	TIMESTAMP
checkout_at	TIMESTAMP
status	ENUM
verified_by	UUID

Field	Type
verified_at	TIMESTAMP

Status:

attended

no_show

excused

cancelled

Attendance is authoritative system data.

37. tasks

Tracks work completed during an event.

Field	Type
id	UUID
event_id	UUID
assignment_id	UUID
description	TEXT
status	ENUM
completed_at	TIMESTAMP
verified_by	UUID

38. reliability_records

Stores reliability inputs/outcomes.

Field	Type
id	UUID
volunteer_id	UUID
event_id	UUID
assignment_id	UUID
outcome	TEXT

Field	Type
cancellation_notice_hours	NUMERIC
created_at	TIMESTAMP

Reliability should be calculated from:

- Completed assignments
- No-shows
- Cancellations
- Notice provided
- Recent participation
- Task completion
- Participation consistency
- Punctuality where tracked

New volunteers should receive:

Not enough history

rather than being penalized.

39. karma_transactions

Karma must be an append-only ledger.

Field	Type
id	UUID
volunteer_id	UUID
event_id	UUID
attendance_id	UUID
points	INTEGER
reason	TEXT
created_at	TIMESTAMP

Important rules:

- No direct user editing.
- No duplicate transaction for the same verified attendance.

- No Karma without verified attendance.
- Corrections use compensating transactions rather than editing old transactions.

Example:

Attendance verified

↓

Event completed

↓

Karma transaction

↓

Volunteer total

40. achievements

Stores achievement definitions.

Field	Type
id	UUID
name	TEXT
description	TEXT
criteria	JSONB
active	BOOLEAN

41. volunteer_achievements

Links volunteers to achievements.

Field	Type
id	UUID
volunteer_id	UUID
achievement_id	UUID
earned_at	TIMESTAMP

42. emergency_requests

Emergency operations should be event-linked.

Field	Type
id	UUID
ngo_id	UUID
event_id	UUID
title	TEXT
description	TEXT
geo	GEOGRAPHY
required_skills	TEXT[]
headcount	INTEGER
urgency	ENUM
status	ENUM
created_by	UUID
created_at	TIMESTAMP

43. emergency_broadcasts

Records emergency broadcasts.

Field	Type
id	UUID
emergency_id	UUID
sent_by	UUID
sent_at	TIMESTAMP
target_filter	JSONB
recipient_count	INTEGER

44. emergency_responses

Stores volunteer responses.

Field	Type
id	UUID
broadcast_id	UUID
volunteer_id	UUID
response	ENUM
responded_at	TIMESTAMP
is_backup	BOOLEAN
deployed_at	TIMESTAMP
deployed_by	UUID
checked_in_at	TIMESTAMP

Important:

Alerted

↓

Confirmed

↓

Deployed

These are three separate states.

A volunteer confirming availability does NOT mean they have been deployed.

45. NGO Transparency

ngo_transparency_records

Field	Type
id	UUID
ngo_id	UUID
type	TEXT
content	JSONB/TEXT
source	ENUM
verification_status	ENUM

Field	Type
created_at	TIMESTAMP

Source:

platform

ngo

NGO-provided information must be clearly labelled.

46. NGO Indicators

ngo_indicators

Stores derived transparency/activity indicators.

Field	Type
ngo_id	UUID
activity_indicator	TEXT
completion_indicator	TEXT
participation_indicator	TEXT
consistency_indicator	TEXT
information_completeness	TEXT
computed_at	TIMESTAMP

These should be calculated from platform activity where possible.

Do not turn this into an opaque "NGO rating."

The product should explicitly state:

Platform-derived indicators based on activity and information available on SevaSetu. They are not a certification, legal verification, or guarantee of an NGO's legitimacy or use of donations.

47. notifications

Field	Type
id	UUID
user_id	UUID
type	TEXT

Field	Type
payload	JSONB
channel	ENUM
status	ENUM
created_at	TIMESTAMP
read_at	TIMESTAMP

Channels:

in_app

email

Emergency alerts should prioritize in-app realtime delivery.

48. Agent Data Model

Firebase should contain:

agentSessions

sessionId

userId

role

ngold

eventId

mode

createdAt

status

agentMessages

messageId

role

content

timestamp

agentContext

sessionId
eventSummary
historicalSummary
sourceIds
builtAt
expiresAt

agentActivity

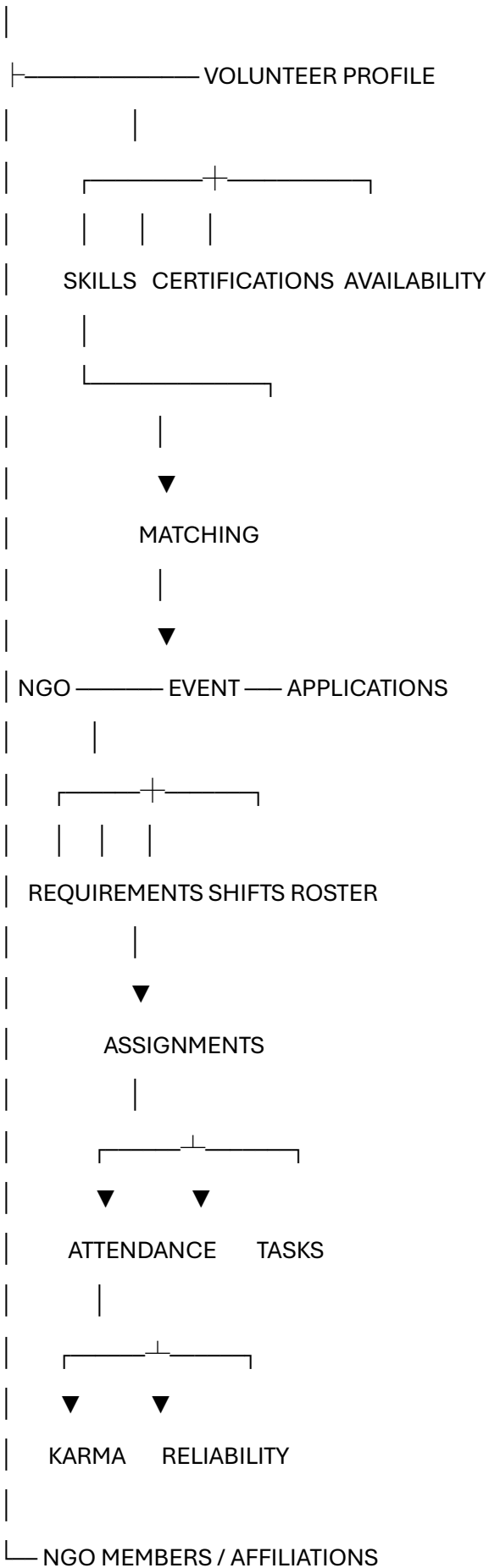
activityId
sessionId
actionType
model
latency
status
timestamp

agentOutputs

outputId
sessionId
type
content
basis
disclaimer
humanAction
Human action:
none
accepted
dismissed

49. Database Relationship Overview

USERS



50. API Folder Responsibility

Each backend module should follow:

module/

└─ controller.js

└─ service.js

└─ repository.js

└─ routes.js

└─ schema.js

└─ tests/

Example:

matching/

└─ controller.js

└─ matching.service.js

└─ matching.repository.js

└─ matching.routes.js

└─ matching.schema.js

└─ tests/

Controller

Handles HTTP requests.

Service

Contains business logic.

Repository

Handles database interaction.

Schema

Validates incoming data.

This prevents database logic from becoming mixed with API code.

51. Matching Architecture

Matching should NOT be an LLM decision.

Use deterministic code.

Event Requirements



Candidate Retrieval



Eligibility Gates



Skill/Certification



Availability



Proximity



Experience



Reliability



Weighted Score



Ranked Results



Explanation



Human Confirmation

The Agent can rephrase the explanation but cannot change the rank or eligibility decision.

52. Scheduling Architecture

MVP scheduling can use a greedy algorithm.

Requirements



Candidates



Highest-ranked available volunteer



Check existing assignments



Check overlap



Check workload



Assign



Repeat



Allocate backups



Generate conflict report



Draft roster



Human finalization

An optimization solver is unnecessary for the hackathon MVP.

53. Emergency Architecture

Authorized NGO User



Create Emergency



Backend Authorization



Save to Supabase



Find Eligible Volunteers

↓

Realtime Firebase Alert

↓

Volunteer Confirm / Decline

↓

Live Count

↓

Backup Candidates

↓

Human Approval

↓

Coordinator Deploys

↓

Check-in

↓

Attendance

↓

Karma + Reliability

Firebase is only the realtime layer.

Supabase remains authoritative.

54. Event Intelligence Agent Flow

User

↓

POST /agent/sessions/:id/messages

↓

Clerk Authentication

↓

Role + NGO Scope Verification

↓

Context Builder

↓
Relevant Supabase Data
↓
Minimal Context
↓
Python Agent Service
↓
Primary LLM
↓
Output Validation
↓
Firebase Session/Output State
↓
Response
↓
Human Decision

55. Agent Context Rules

For volunteer mode:

The Agent can receive:

- Event details
- Event requirements
- Public NGO information
- Preparation information

It must NOT receive:

- Other volunteers' private information
- Private NGO information
- Unnecessary personal information

For NGO mode:

The Agent can receive:

- Event information

- NGO information
 - Aggregated historical event statistics
 - Permitted operational context
-

56. Maps API Architecture

The Maps API should provide:

- Geocoding
- Event location
- Distance calculation
- Location-based discovery

The platform should store coordinates in Supabase/PostGIS.

Distance should be calculated server-side.

Never send raw volunteer coordinates to:

- Other volunteers
- Public users
- NGOs unnecessarily

Expose only:

Nearby

Within 5 km

Within 10 km

or similar distance bands.

57. Notification Architecture

Primary:

In-app notifications

Fallback:

Email API

Automation:

n8n

Example:

Roster finalized

↓

Create notification

↓

In-app notification

↓

Email fallback if required

Emergency:

Emergency created

↓

Realtime in-app alert

↓

Email fallback

SMS/WhatsApp are not required for MVP.

58. Environment Variables

Frontend

VITE_CLERK_PUBLISHABLE_KEY=

VITE_API_BASE_URL=

VITE_MAPS_PUBLIC_KEY=

Never put private API keys in the frontend.

59. Node/Express API Environment Variables

NODE_ENV=development

PORT=5000

FRONTEND_URL=

CLERK_SECRET_KEY=

CLERK_WEBHOOK_SECRET=

SUPABASE_URL=

SUPABASE_ANON_KEY=

SUPABASE_SERVICE_ROLE_KEY=

FIREBASE_PROJECT_ID=

FIREBASE_CLIENT_EMAIL=

FIREBASE_PRIVATE_KEY=

MAPS_API_KEY=

EMAIL_API_KEY=

EMAIL_FROM=

N8N_WEBHOOK_URL=

AGENT_SERVICE_URL=

AGENT_SERVICE_SECRET=

PRIMARY_LLM_PROVIDER=

PRIMARY_LLM_API_KEY=

FALLBACK_LLM_PROVIDER=

FALLBACK_LLM_API_KEY=

LOG_LEVEL=info

60. Python Agent Environment Variables

ENVIRONMENT=development

PORT=8000

AGENT_SERVICE_SECRET=

FIREBASE_PROJECT_ID=

FIREBASE_CLIENT_EMAIL=

FIREBASE_PRIVATE_KEY=

PRIMARY_LLM_PROVIDER=

PRIMARY_LLM_API_KEY=

FALLBACK_LLM_PROVIDER=

FALLBACK_LLM_API_KEY=

MODEL_NAME=

MAX_CONTEXT_SIZE=

MAX_AGENT_TOKENS=

AGENT_RATE_LIMIT=

LOG_LEVEL=info

The Python service should not receive the Supabase service-role key.

The Agent should receive scoped context from Express instead.

61. Supabase Configuration

Enable:

PostgreSQL

PostGIS

Row-Level Security

Recommended indexes:

events(ngo_id)

events(status)

events(start_time)

events(visibility)

applications(event_id)

applications(volunteer_id)

assignments(event_id)

assignments(volunteer_id)

assignments(shift_id)

attendance(volunteer_id)

attendance(assignment_id)

karma_transactions(volunteer_id)

reliability_records(volunteer_id)

ngo_members(ngo_id)

ngo_members(user_id)

ngo_affiliations(ngo_id)

ngo_affiliations(volunteer_id)

For location:

Use PostGIS indexes on:

volunteer_profiles.home_geo

events.geo

62. Database Constraints

Important constraints:

Users

clerk_user_id UNIQUE

Applications

(event_id, volunteer_id) UNIQUE

NGO membership

(ngo_id, user_id) UNIQUE

NGO affiliation

Avoid duplicate active affiliations.

Karma

Use a uniqueness/idempotency mechanism based on:

attendance_id

to prevent duplicate awards.

Assignments

Prevent duplicate active assignment for the same volunteer and shift.

63. RLS Strategy

Supabase RLS should provide defense in depth.

Examples:

Volunteer profile

A volunteer can update:

their own profile

but not:

another volunteer's profile

NGO

NGO members can access:

their NGO

but not:

another NGO

Events

NGO staff can modify:

events belonging to their NGO

Attendance

Users cannot arbitrarily edit attendance.

Only authorized backend workflows should modify authoritative attendance records.

Karma

Volunteers can read their Karma.

They cannot insert/update/delete Karma transactions.

64. Security Architecture

Clerk

↓

Authentication

↓

Express Middleware

↓

Role Verification

↓

NGO Scope Verification

↓

Input Validation

↓

Business Logic

↓

Supabase RLS

↓

Database

For Agent:

Express

↓

Permission Check

↓

Minimal Context

↓

Python Agent

↓

LLM

The Agent never gets unrestricted database write access.

65. Environment Separation

Maintain separate environments:

development

staging

production

Each environment should have separate:

- Clerk configuration
- Supabase project/database
- Firebase project
- API keys
- LLM keys
- Email configuration

Never use production credentials during development.

66. Configuration Files

Recommended:

.env

.env.local

.env.example

Never commit:

.env

.env.local

Commit:

.env.example

with empty values.

67. Matching Configuration

Do not hard-code matching weights throughout the application.

Create:

packages/matching-config/

Example:

```
{
  skills: 35,
  availability: 20,
  proximity: 20,
  experience: 15,
  reliability: 10
}
```

Store the active configuration with each match_run.

This makes matching auditable.

68. Karma Configuration

Keep Karma rules configurable.

Example:

```
{
  pointsPerVerifiedHour: 10,
  completionBonus: 20
}
```

Milestone configuration:

```
{
  100: "Community Starter",
  500: "Active Contributor",
  1000: "Community Champion"
}
```

The exact values should be confirmed before implementation.

69. Reliability Configuration

Keep:

- Minimum history threshold
- Recency weighting

- No-show penalty
- Cancellation treatment
- Task completion weight

in configuration rather than scattered constants.

The UI should display:

Not enough history

until enough evidence exists.

70. API Error Format

Use a consistent response format.

Example:

```
{  
  "success": false,  
  "error": {  
    "code": "EVENT_CAPACITY_REACHED",  
    "message": "This event has reached its volunteer capacity."  
  },  
  "requestId": "req_123"  
}
```

Do not expose:

database errors

stack traces

API keys

internal service details

to users.

71. Logging

Log:

- Request ID
- User ID
- Endpoint

- Response status
- Duration
- Error code

Sensitive actions should additionally log:

- Actor
- Target record
- Action
- Timestamp

Examples:

MATCH_RUN

ROSTER_FINALIZED

ATTENDANCE_VERIFIED

KARMA_AWARDED

EMERGENCY_CREATED

EMERGENCY_ALERTED

EMERGENCY_DEPLOYED

72. Performance Strategy

For the MVP:

Event discovery

Use database filtering and indexes.

Matching

First filter candidates.

Do not calculate every criterion for every volunteer in the entire database.

All Volunteers

↓

Active

↓

Event Eligibility

↓

Skill/Certification

↓

Availability

↓

Distance

↓

Final Scoring

Scheduling

Process per event.

Agent

Run independently so an LLM slowdown does not block the core application.

73. Scalability Strategy

The architecture is designed around stateless API servers.

Frontend

↓

Load-balanced API

↓

Supabase

Agent service can scale separately:

API

↓

Agent Service

↓

Multiple Agent workers

Matching and roster generation are naturally event-scoped.

This makes them easier to scale later.

74. Failure Handling

Agent unavailable

Core application continues.

Agent unavailable

↓

Show temporary unavailable message

↓

Events/matching/attendance continue

Firestore unavailable

Emergency realtime UI can fall back to API polling.

Supabase remains authoritative.

Email unavailable

In-app notifications continue.

Maps unavailable

Distance ranking can temporarily be unavailable.

The platform should not fabricate distances.

LLM unavailable

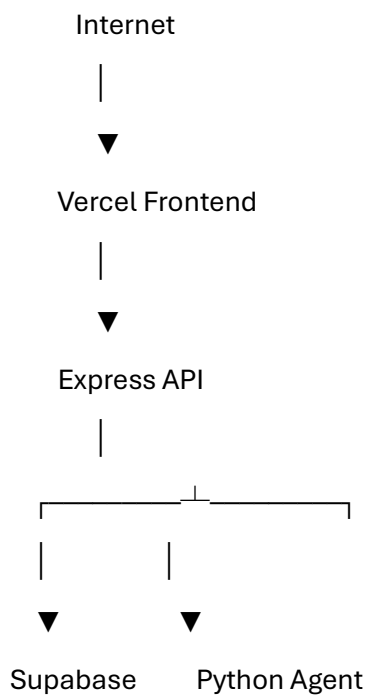
Use fallback model if configured.

Otherwise show:

Agent temporarily unavailable.

75. Deployment Architecture

Recommended:



|



LLM Provider

Supporting services:

Clerk

Firebase

Maps API

Email API

n8n

The Python Agent should run on a suitable long-running/containerized environment rather than being forced into a serverless environment if its workload requires longer execution.

76. Recommended Development Order

Phase 0 — Setup

Build:

- Repository
- React/Vite
- Express
- Clerk
- Supabase
- Firebase
- Environment configuration

Phase 1 — Profiles

Build:

- Authentication
- Volunteer profile
- Skills
- Certifications
- Availability
- NGO profile

- NGO members
-

Phase 2 — Events

Build:

- Event CRUD
 - Requirements
 - Shifts
 - Public discovery
 - Registration
 - Maps/distance
-

Phase 3 — Matching & Rostering

Build:

- Eligibility gates
 - Matching score
 - Explainability
 - Match audit
 - Roster generation
 - Conflict detection
 - Backup allocation
-

Phase 4 — Tracking

Build:

- Check-in
 - Attendance verification
 - Task completion
 - Karma
 - Reliability
 - Verified history
-

Phase 5 — Transparency + Agent

Build:

- NGO transparency page
 - Activity indicators
 - Agent service
 - Agent chat
 - Event preparation
 - Risk suggestions
 - Historical analysis
-

Phase 6 — Demo Polish

Build:

- Seed data
 - Dashboard
 - UX improvements
 - Loading states
 - Error states
 - Demo rehearsal
-

Phase 7 — Emergency

Build:

- Emergency creation
- Real-time alert
- Confirmation
- Live count
- Backup suggestions
- Deployment
- Emergency check-in

This should remain secondary to the primary golden journey for the hackathon.

77. MVP Cut Order

If development time becomes limited, cut features in this order:

1. Advanced analytics



2. Burnout indicators



3. Advanced emergency features



4. Advanced notification workflows



5. Advanced backup logic



6. Hugging Face features



7. UI polish

Protect the primary demo:

Profile



Event



Requirements



Matching



Explanation



Roster



Attendance



Karma



Transparency

↓

Agent

↓

Dashboard

78. Hackathon Demo Architecture

The primary demonstration should prove the complete platform loop.

Volunteer Profile

↓

NGO Creates Event

↓

Event Requirements

↓

Matching Engine

↓

"Why this volunteer?"

↓

Roster Generation

↓

Volunteer Confirmation

↓

Attendance

↓

Karma

↓

Reliability

↓

NGO Transparency

↓

Event Intelligence Agent

↓

Impact Dashboard

This demonstrates the strongest differentiators in a single flow.

79. What NOT to Build in the MVP

Do not allow scope creep into:

- Government scheme integration
- Karma-based government benefits
- Guaranteed NGO employment
- Guaranteed NGO positions
- Legal NGO certification
- Autonomous emergency deployment
- Autonomous AI decision-making
- SMS/WhatsApp integration
- Advanced ML matching
- Full payment processing
- Complex billing
- Large-scale predictive analytics

The PRD explicitly keeps these outside the MVP/future scope.

80. Important Product Architecture Decisions

Decision 1

Event is the central entity.

Everything operational should connect to an event.

Decision 2

Karma and Reliability are separate.

Karma

= Contribution

Reliability

= Commitment consistency

Karma should not directly influence matching.

Decision 3

Independent volunteers do not require NGO membership.

They can:

Discover public event

↓

Register

↓

Participate

↓

Verified attendance

↓

Karma

↓

Build history

↓

Potential NGO affiliation

Decision 4

NGO Transparency is not an NGO rating.

The platform provides indicators.

It does not certify an NGO or declare an NGO good/bad.

Decision 5

Agent is intelligence, not authority.

Agent recommends

Human decides

Backend enforces

Database records

Decision 6

Supabase is authoritative.

Firebase is:

Agent state

+

Realtime state

+

Temporary context

81. Architecture Definition of Done

The SevaSetu MVP architecture is correctly implemented when:

Frontend

- React/Vite application works.
- Role-based dashboards exist.
- Public event discovery works.
- Agent UI exists.

Authentication

- Clerk authentication works.
- Protected routes work.
- Role checks work.

Backend

- Express REST API works.
- API validation exists.
- Authorization exists.
- NGO scope checking exists.

Database

- Supabase is the authoritative store.
- RLS is enabled.
- Event-centered relationships work.
- Matching results are auditable.

Matching

- Eligibility gates work.
- Weighted ranking works.
- Explanations are generated from actual criteria.
- Karma is excluded from matching.

Scheduling

- Conflicts are detected.
- Draft roster is generated.
- Human finalization is required.

Tracking

- Attendance is verified.
- Karma is ledger-based.
- Reliability is separate.
- History is derived from verified records.

Transparency

- NGO indicators are visible.
- Platform-derived and NGO-provided information are separated.
- Disclaimer is displayed.

Agent

- Agent receives scoped context.
- Agent cannot directly modify core records.
- Agent outputs are labelled as suggestions.
- Agent failure does not break core platform operations.

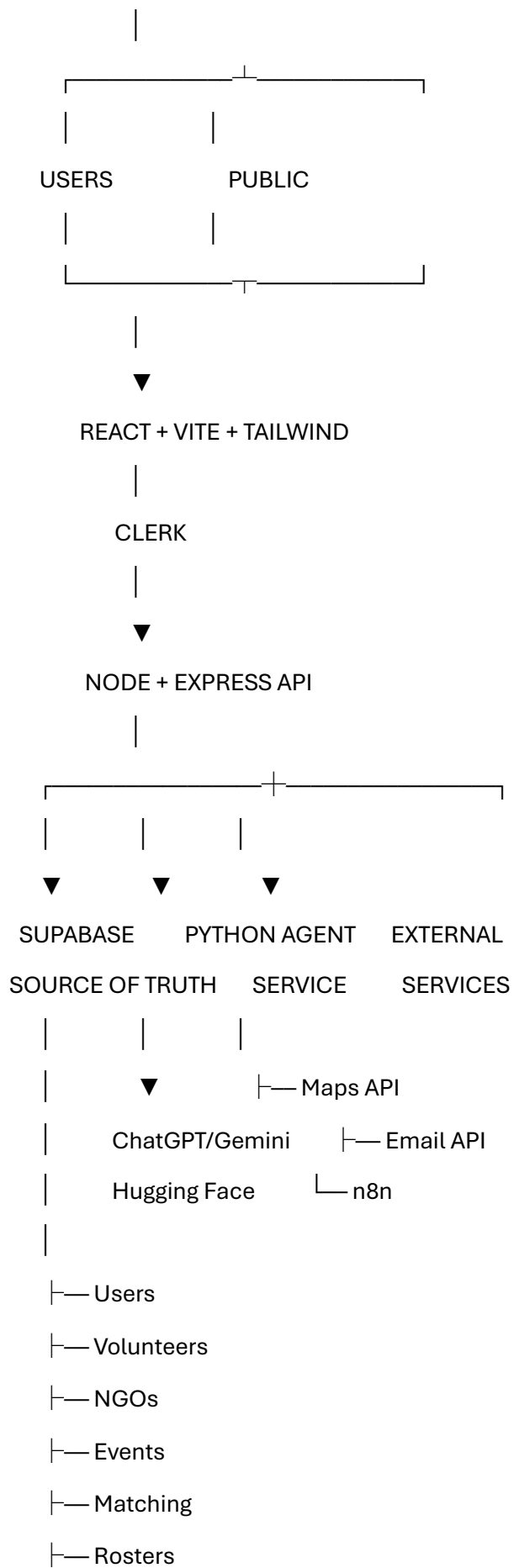
Emergency

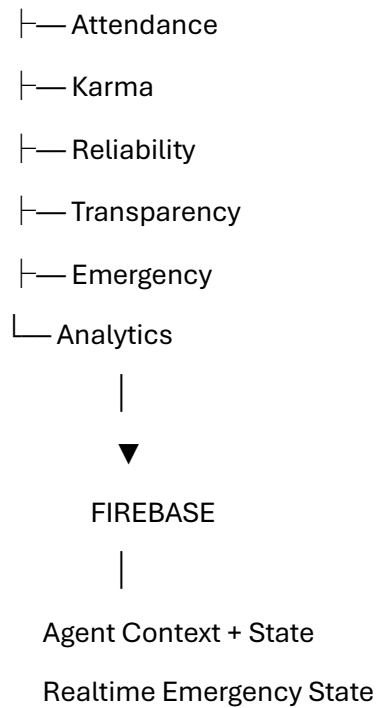
- Authorization is enforced.
- Alert, confirmation and deployment are separate.
- Supabase remains authoritative.
- Firebase provides realtime state.

82. Final Recommended Architecture

The complete SevaSetu architecture can be summarized as:

SEVASETU





Final Architectural Principle

SevaSetu should be a deterministic operational platform with an AI intelligence layer — not an AI system with a platform attached to it.

The core system should remain reliable even if the AI, Maps API, Email API or Firebase temporarily fails.

The authoritative chain should always be:

User

↓

Clerk

↓

Express Authorization

↓

Deterministic Business Logic

↓

Supabase

while AI operates as:

Relevant Supabase Context

↓

Python Agent

↓

LLM

↓

Recommendation

↓

Human Decision

↓

Normal SevaSetu Workflow

That separation gives SevaSetu a much stronger architecture for the hackathon: the AI is meaningful and demonstrable, but the critical operations remain explainable, auditable and under human control.